

Chapter 7

CSV Data Analysis: Your First Small Biology Project

Read real table-shaped data, calculate DNA features, filter samples, and make a graph.

EDOS learning idea

We will use biology and DNA examples because they are concrete, visual, and easy to experiment with. Type the examples yourself, change one thing, run again, and observe what changed.

What you will be able to do

- Understand what a CSV file is.
- Read CSV data with Python.
- Use pandas for beginner-friendly table analysis.
- Create new columns from DNA sequences.
- Filter and summarize biological samples.
- Make a simple Matplotlib visualization in Jupyter.

Classroom rhythm

Read a small idea -> run the example -> change one value -> predict the result -> run again -> explain what happened in your own words.

7.1 What is a CSV file?

CSV means comma-separated values. It is a plain-text table format. Each line is one row, and commas separate columns. CSV is common because spreadsheet programs and programming languages can both read it.

```
sample_id,species,sequence
S01,E_coli,ATGCGT
S02,Yeast,GGGCCC
```

In EDOS, keep CSV files in the dataset folder and open analysis notebooks from Jupyter.

7.2 Reading CSV with the standard library

```
import csv

with open('datasets/dna_samples.csv', newline='') as f:
    reader = csv.DictReader(f)
    rows = list(reader)

print(rows[0])
```

csv.DictReader turns each row into a dictionary. This works without installing pandas.

7.3 pandas: tables with less typing

```
import pandas as pd

df = pd.read_csv('datasets/dna_samples.csv')
print(df.head())
print(df.columns)
```

A pandas DataFrame is a table object. Think of it as a spreadsheet that Python can manipulate.

7.4 Create sequence length and GC columns

```
def gc_percent(dna):
    dna = dna.upper()
    return (dna.count('G') + dna.count('C')) / len(dna) * 100

df['length'] = df['sequence'].str.len()
df['gc_percent'] = df['sequence'].apply(gc_percent)
print(df[['sample_id', 'length', 'gc_percent']])
```

This is a powerful idea: write a function once, then apply it to every sequence in a table.

7.5 Filter samples

```
high_gc = df[df['gc_percent'] >= 50]
print(high_gc[['sample_id', 'species', 'gc_percent']])
```

The expression inside square brackets creates a True/False mask. pandas keeps only rows where the condition is True.

7.6 Group and summarize

```
summary = df.groupby('species')['gc_percent'].mean()
print(summary)
```

groupby() collects rows by a category, then mean() calculates the average for each group. This is a common data-analysis pattern.

7.7 Plot the result in Jupyter

```
import matplotlib.pyplot as plt

plt.bar(df['sample_id'], df['gc_percent'])
plt.axhline(50, linestyle='--')
plt.title('GC percentage by DNA sample')
plt.xlabel('Sample')
plt.ylabel('GC %')
plt.show()
```

In EDOS, Matplotlib figures are designed to appear directly under the Jupyter cell.

7.8 Final mini-project

Your final task is to load dna_samples.csv, calculate sequence length and GC percentage, identify the highest-GC sample, summarize average GC by species, and create one graph. Then write two Markdown sentences explaining what you noticed.

That is already a real analysis workflow: load -> clean/check -> calculate -> summarize -> visualize -> explain.

Try it yourself

These are practice prompts, not a test. If your code breaks, that is useful information. Read the error, change one thing, and try again.

1. Open `dna_samples.csv` and inspect the first rows.
2. Add `length` and `gc_percent` columns.
3. Find the sample with the highest GC percentage.
4. Filter samples longer than 24 bases.
5. Group by species and calculate mean sequence length.
6. Create a bar chart and write a short Markdown interpretation.

Tiny debugging habit

When something fails, check spelling, quotation marks, brackets, colons, and indentation first. Then print intermediate values so you can see what Python sees.

Chapter summary

- CSV stores table-shaped data.
- The `csv` module is built into Python.
- `pandas` makes table analysis easier.
- Functions can be applied to `DataFrame` columns.
- Filtering and grouping answer biological questions.
- A good analysis ends with an explanation, not just a graph.

Next step

Open Notebook 07 and complete the final DNA CSV analysis. Save the notebook as your first mini biology data project.

EDOS - Educational Desktop Operating Simulator
Copyright by Dr. Yash Munnalal Gupta